FlowForge — Workflow Orchestrator Documentation

Oct 19, 2026 · @Luke

1. Overview

FlowForge runs business workflows that you draw on a grid, and starts them from HTTP calls, RabbitMQ messages, MQTT events or a schedule. Each running workflow lives in its own Node.js process until it ends, so a slow or failing workflow never affects the others.

It ships as two applications built from one codebase:

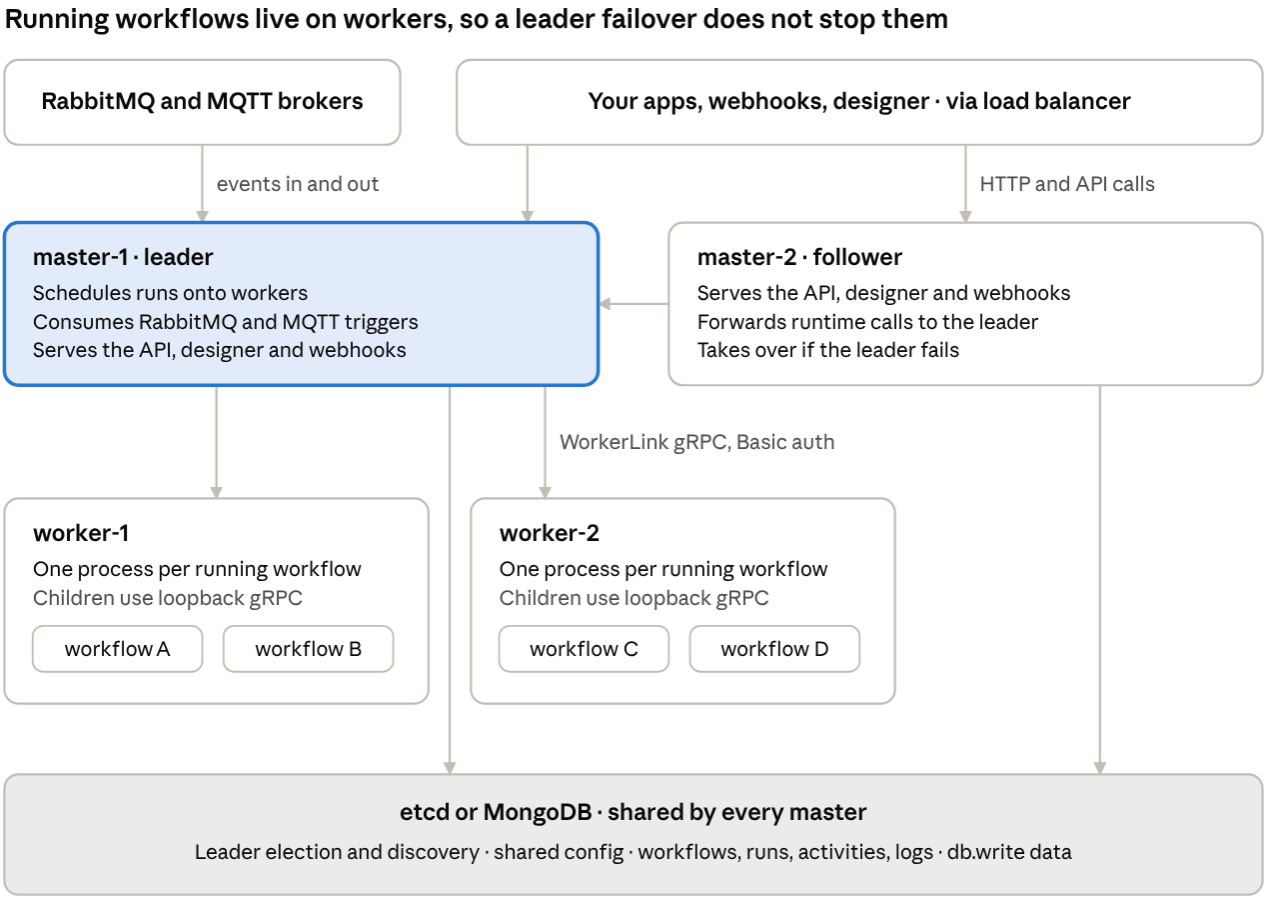
Application	Command	What it does
Master	`node bin/master.js --config config/master-1.yaml`	REST API, the visual designer, triggers, workflow and instance storage, scheduling. Run two or more for high availability.
Worker	`node bin/worker.js --config config/worker-1.yaml`	Only hosts workflow processes (one forked process per running workflow). Add workers to scale out.

What you can do with it:

- Design workflows in a browser with drag-and-drop nodes: Activity (API call, RabbitMQ/MQTT publish, receive an event, database write, transform), Function, Decision, Switch, Join, Response, Timer, Delay, Signal, Sub-workflow and End.
- Start workflows from an HTTP webhook, the start API, a RabbitMQ queue, an MQTT topic or a schedule, with an auto-generated or caller-supplied workflow ID.
- Stop, terminate, pause, resume and signal running workflows, retry failed ones, and see every step's input, output and error.
- Keep the incoming request data, an activity history and logs with a configurable retention period.
- Run several masters and workers as a cluster, with leader election, automatic failover and all shared state in etcd or MongoDB.

2. Architecture: how it works

Masters decide what runs where; workers run it; etcd or MongoDB holds everything the cluster shares. Because a workflow's process lives on a worker, not on a master, a master can fail without stopping it.



FlowForge cluster · 2 masters, 2 workers, shared etcd or MongoDB

One master is elected leader and does the scheduling; the follower answers the same API and takes over if the leader fails. Add workers to run more workflows at once.

Life of one workflow run

1. A trigger arrives at any master: a webhook, a start API call, or (on the leader) a RabbitMQ or MQTT message.
2. The run is saved as QUEUED with its input and the incoming request data. A follower forwards the call to the leader over gRPC.
3. The leader picks the least-loaded worker whose tags match the workflow's `settings.workerTags` and tells it to start the run.
4. The worker forks a new Node.js process for that run. The process connects back to the worker over loopback gRPC, using the instance ID and a one-time token as its Basic credentials.

5. The process executes the graph. Every status change, step result and log line goes to the worker, which relays it to the leader, which saves it.
6. When a node needs a broker, the data database or a sub-workflow, the process asks the leader through the same channel; masters own all external connections.
7. Stop, terminate, pause, resume and signal commands go from the leader through the worker to the process.
8. When the workflow ends, its final status and output are saved and the process exits.

Internal gRPC

All internal traffic uses gRPC with HTTP Basic credentials in the `authorization` header, checked in constant time. TLS can be turned on.

Service	From → to	Credentials	Carries
WorkerLink.Attach	Worker → master	Cluster username and password	Registration, start and kill commands, relayed workflow traffic, heartbeats. Non-leader masters answer with a redirect to the leader.
Peer.Invoke, Peer.Watch	Follower → leader	Cluster username and password	Forwarded start, control and wait calls; live updates for the follower's designer
Orchestrator.Connect	Workflow process → its worker (loopback)	Instance ID and a per-process token	Start command, controls, status, steps, logs, requests and replies

Leader election

Masters compete for a leader record in etcd (a key on a lease) or MongoDB (a document with an expiry). Each new leader gets a higher term number; workers ignore a leader with an older term, and a master that loses its lease stops writing at once.

3. Installation and quick start

You need Node.js 18 or newer. A single machine needs nothing else; a cluster also needs etcd or MongoDB, and real brokers (RabbitMQ, an MQTT broker) when you use those triggers.

Single machine

```
npm install
npm start          # master + built-in worker, file store (config/single-
node.yaml)
```

Open `http://localhost:3000` for the designer. Four example workflows are loaded into an empty store: `order-approval`, `sensor-alert`, `payment-saga` and `device-telemetry`.

Local cluster (2 masters, 2 workers, etcd on 127.0.0.1:2379)

```
export FLOWFORGE_GRPC_PASSWORD=$(openssl rand -hex 16)   # set on every node
node bin/flowforge.js config push config/cluster.yaml --config config/master-
1.yaml
node bin/master.js --config config/master-1.yaml   # HTTP 3000, gRPC 50051
node bin/master.js --config config/master-2.yaml   # HTTP 3001, gRPC 50052
node bin/worker.js --config config/worker-1.yaml
node bin/worker.js --config config/worker-2.yaml
curl localhost:3001/api/cluster                    # one leader, both workers
connected
```

Docker Compose

```
FLOWFORGE_GRPC_PASSWORD=$(openssl rand -hex 16) docker compose up --build
```

This starts etcd, RabbitMQ, Mosquitto, MongoDB, two masters behind nginx on port 8080 and two workers. A one-off `config-init` service pushes `config/cluster.yaml` into etcd first.

Tests

```
npm test                                          # 18 end-to-end tests: master
+ separate worker
ETCD_BIN=/path/to/etcd npm run test:cluster     # 14 cluster tests incl.
leader kill -9, worker loss, drain
```

4. Configuration

In cluster mode, settings shared by all nodes live in one document in etcd or MongoDB; each node's own YAML file holds only its identity, ports and where that document is. A node's effective settings are built in this order, later wins: built-in defaults, then the shared `common` section, then the shared `master` or `worker` section, then the node's own YAML.

Where everything is stored

What	Cluster mode	Single node
Cluster-wide settings	Shared config document in etcd or MongoDB	The one YAML file
Workflows, instances, activities, request data, logs	Store: etcd or MongoDB	Store: file or memory
Rows written by the `db.write` activity	MongoDB, or etcd keys	JSON files allowed
Leader record, list of masters and workers	Coordinator: etcd or MongoDB	Not needed
Node ID, ports, advertised address, TLS files	The node's own YAML	The one YAML file

Cluster mode refuses local files for the store or `db.write` data, so nothing important is ever kept on one machine.

A node's own YAML

```
node: { id: master-1 }
http: { port: 3000, advertiseUrl: http://10.0.0.11:3000 }
grpc: { port: 50051, advertiseAddress: 10.0.0.11:50051 }
shared:
  source: etcd                        # or mongo
  etcd: { hosts: ["10.0.0.5:2379"], prefix: /flowforge/ }
  # mongo: { url: mongodb://10.0.0.6:27017, db: flowforge }
```

A worker's YAML is the same shape plus `node.tags` (e.g. `[default, eu]`) and, optionally, local overrides such as `runtime.maxConcurrent` for a bigger machine.

The shared document (`config/cluster.yaml`)

```
common:
  security:
    grpc: { username: flowforge, password: ${FLOWFORGE_GRPC_PASSWORD} }
master:
  cluster: { enabled: true, coordinator: etcd, leaseTtlMs: 6000,
workerLostGraceMs: 15000, onWorkerLost: fail }
  store: { type: etcd, etcd: { hosts: ["${ETCD_HOSTS:-127.0.0.1:2379}"],
prefix: /flowforge/ } }
  dataDb: { type: auto, mongoUrl: ${DATA_MONGO_URL:-} }
  rabbitmq: { url: ${RABBITMQ_URL:-} }
  mqtt: { url: ${MQTT_URL:-}, sharedGroup: flowforge }
  logs: { retentionHours: 168 }
worker:
  discovery: { type: etcd, etcd: { hosts: ["${ETCD_HOSTS:-127.0.0.1:2379}"],
prefix: /flowforge/ } }
  runtime: { maxConcurrent: 50, drainTimeoutMs: 60000 }
```

`${VAR}` and `${VAR:-default}` stay as text in etcd/MongoDB and each node fills them in from its own environment, so passwords never need to be stored there.

Admin commands

```
node bin/flowforge.js config validate config/cluster.yaml
node bin/flowforge.js config push config/cluster.yaml --config config/master-
1.yaml   # or --etcd host:2379 / --mongo <url>
node bin/flowforge.js config pull --config config/master-1.yaml [--reveal] [-
-out file.yaml]
node bin/flowforge.js config effective --config config/worker-1.yaml   #
merged settings, secrets hidden
```

- Every push is checked first, and refused if someone else changed the document after you pulled it (`--if-version N`; `--force` overrides).
- Pulled output hides secrets as `***`; pushing a hidden value back is refused.
- Running nodes watch the document. Changes to credentials, API login, log level and retention, worker-lost timing, terminate grace and worker capacity apply immediately; anything else is logged as "restart required".
- The same document can be read and edited over HTTP: `GET/PUT /api/cluster/config` (editing requires API login to be enabled).

Most-used settings

Setting	Default	Meaning
`security.grpc.username` / `password`	must be set	Basic credentials every worker and master presents on internal gRPC
`security.api.enabled`	false	Basic login for the REST API and designer (`protectHooks` also covers webhooks)
`grpc.tls.enabled`	false	TLS for internal gRPC; turn on across untrusted networks
`cluster.leaseTtlMs`	10000	Leader lease; failover takes about one lease
`cluster.workerLostGraceMs`	15000	Time a disconnected worker has to reconnect
`cluster.onWorkerLost`	fail	`fail` marks its workflows FAILED; `restart` re-runs them from the start on another worker
`dataDb.type`	auto	`mongo` , `etcd` or `file` (single node only) for `db.write`
`logs.retentionHours`	168	How long logs are kept
`runtime.maxConcurrent` (worker)	50	Workflow processes one worker accepts
`runtime.childMaxOldSpaceMb`	256	Memory limit per workflow process
`runtime.drainTimeoutMs` (worker)	60000	On SIGTERM, how long to wait for running workflows

5. Designing workflows

A workflow is a graph of nodes joined by connections; you build it in the designer or send it as JSON, and FlowForge checks it before saving.

Using the designer

1. Pick a workflow from the dropdown, or press **New** and give it an ID.
2. Drag nodes from the left palette onto the grid (they snap to a 20 px grid), or click one to drop it in the centre.

3. Drag from a node's coloured output dot on its right edge to another node to connect them. Named outputs (`true`, `false`, `error`, `timeout`, switch cases) each have their own dot.
4. Click a node to edit its settings in the right-hand panel; the fields shown change with the node's options. With nothing selected, the panel edits the workflow itself and its triggers.
5. **Validate**, **Save** (Ctrl+S), then **Run** to start it with test input.

Other controls: Delete removes the selection, Ctrl+D duplicates a node, the mouse wheel zooms, Alt or Space plus drag pans. The **Instances** drawer at the bottom lists runs live; selecting one colours each node by its result and shows activities, logs, request data and output. **Cluster** shows the masters and workers.

Workflow JSON

```
{
  "id": "order-workflow",
  "name": "Order workflow",
  "enabled": true,
  "triggers": [ { "type": "http", "method": "POST", "path": "/orders",
"instanceIdFrom": "body:orderId" } ],
  "settings": { "maxSteps": 1000, "workerTags": [] },
  "nodes": [
    { "id": "start", "type": "start", "name": "Order in", "x": 40, "y": 140,
"config": {} },
    { "id": "save", "type": "activity", "name": "Save order", "x": 280, "y":
140,
      "config": { "kind": "db.write", "collection": "orders", "document": "{{
input }}" } },
    { "id": "end", "type": "end", "name": "Done", "x": 520, "y": 140,
"config": {} }
  ],
  "edges": [ { "from": "start", "to": "save" }, { "from": "save", "to":
"end", "port": "out" } ]
}
```

Rules checked on save: exactly one Start node with nothing connected into it; every connection uses an output the source node really has; a Switch has at least one case; a Signal has a name; a Sub-workflow names another workflow. `settings.workerTags` limits the workflow to workers carrying all those tags, and `settings.maxSteps` stops runaway loops.

Node reference

Node	Outputs	Settings	What it does
Start	out	`initVars`	Entry point; the trigger data is available as `input`
End	none	`status` (COMPLETED or FAILED), `output`, `error`	Finishes the workflow and cancels other running branches
Activity: `http`	out, error	`method`, `url`, `headers`, `body`, `failOnHttpError`	Calls an API; output `{ status, headers, body }`
Activity: `rabbitmq.publish`	out, error	`exchange`, `routingKey` or `queue`, `message`, `persistent`	Publishes a RabbitMQ message
Activity: `mqtt.publish`	out, error	`topic`, `message`, `qos`, `retain`	Publishes an MQTT message
Activity: `event.receive`	out, timeout, error	`source`, exchange/routing key/queue or topic, `filter`, `timeoutMs`	Waits for a matching RabbitMQ or MQTT event
Activity: `db.write`	out, error	`collection`, `operation` (insert, upsert, update, delete), `keyField`, `document`	Writes to the data database
Activity: `transform`	out, error	`mapping`	Builds a new object from a template
Function	out, error	`code`, `timeoutMs`, `outputVar`	Runs JavaScript; can set `vars`, call `log()`, `fetch`, `sleep`
Decision	true, false	`condition`	If / else on an expression
Switch	one per case, default	`expression`, `cases`	Routes on a value; editing cases adds or removes outputs
Join	out	`count` (0 = all incoming)	Waits for parallel branches, continues once

Node	Outputs	Settings	What it does
Response	out	`statusCode`, `body`, `headers`	Replies to a waiting HTTP caller; the workflow carries on
Timer	out	`mode`: `at` a date, or `interval` with `intervalMs` and `repeat`	Waits until a time, or ticks repeatedly
Delay	out	`duration` (`500`, `30s`, `5m`, `1h30m`)	Pauses this branch
Signal	out, timeout	`mode` (wait or listen), `signalName`, `timeoutMs`	Wait: blocks until the signal arrives. Listen: every arrival interrupts by starting this branch
Sub-workflow	out, error	`workflowId`, `input`, `wait`, `timeoutMs`	Starts another workflow as a child run, in its own process

Every Activity also has `retries`, `retryDelayMs`, `timeoutMs` and `outputVar`. If a node fails and its `error` output is not connected, the whole workflow fails.

Templates and expressions

Any text or JSON setting can contain `{{ expression }}`. A setting that is exactly one placeholder keeps the value's type (`"{{ input.items }}"` stays an array); otherwise the result is inserted as text.

Name	Contains
`input`	The trigger data: HTTP body plus path parameters, or the message body
`vars`	Workflow variables, set by `initVars`, Function code and `outputVar`
`nodes.<nodeId>`	Output of a node that already ran
`signals.<name>`	Last payload received for each signal
`workflow`	`instanceId`, `workflowId`, `name`, `parentInstanceId`, `startedAt`

Name	Contains
`trigger`	How it started: `type`, `headers`, `query`, `params`, `topic`, `routingKey`
`env.WF_*`	Environment variables whose names start with `WF_`, for API keys
`now()`, `uuid()`	Current time in ms, a new UUID

Branching and flow

- One output can connect to several nodes; those branches run at the same time. Use Join to wait for them.
- The workflow completes when an End node runs, or when every branch has finished (output is then `vars`).
- Connect `error` and `timeout` outputs on anything that talks to the outside world, so failures go down a path you designed.

6. Triggers: starting a workflow

A workflow can be started five ways, and one workflow can have several triggers at once. The trigger's data becomes the workflow's `input`.

Trigger	How the caller starts it	Gets a reply?	Set up in
Start API	`POST /api/workflows/<id>/start`	Yes: 202, the Response node's body, or the final result	Nothing to add; works for every workflow
HTTP webhook	`POST /hooks/<path>`	Yes (sync), or 202 straight away	Workflow triggers: `http`
RabbitMQ	Publish to an exchange or queue	No, send the result back as a message	Workflow triggers: `rabbitmq`
MQTT	Publish to a topic	No, send the result back as a message	Workflow triggers: `mqtt`
Schedule	Every `intervalMs`	No	Workflow triggers: `schedule`

Trigger settings

```
"triggers": [
  { "type": "http", "method": "POST", "path": "/orders/:region", "sync":
true, "responseTimeoutMs": 15000, "instanceIdFrom": "body:orderId" },
  { "type": "rabbitmq", "exchange": "shop", "routingKey": "order.created",
"queue": "flowforge.orders", "instanceIdFrom": "body:orderId" },
  { "type": "mqtt", "topic": "orders/+/created", "qos": 1, "instanceIdFrom":
"body:orderId" },
  { "type": "schedule", "intervalMs": 3600000, "input": { "report": "hourly"
} }
]
```

- **HTTP paths** can contain `:name` parameters (merged into `input`) and `*`. With `sync: true` the caller receives the first Response node's reply, or the final result, or 202 after `responseTimeoutMs`.
- **RabbitMQ** uses a durable queue (default `flowforge.<workflowId>`); `*` and `#` work in the routing key. A message is acknowledged only after the run is safely stored.
- **MQTT** topics accept `+` and `#`. In a cluster set `mqtt.sharedGroup` so the broker shares messages between consumers.
- The masters must be connected to the same broker your apps publish to (`rabbitmq.url`, `mqtt.url`). Without a URL, FlowForge uses a built-in emulated bus that only works inside FlowForge.

Workflow instance ID

Each run has an ID: auto-generated (a UUID) or supplied by the caller. Supply it with `instanceIdFrom` (`body:<field.path>`, `header:<name>`, `query:<name>`, `param:<name>`), the `x-workflow-instance-id` header, `?instanceId=`, or `instanceId` in the start API body. IDs are unique: sending the same one again returns 409, and a repeated broker message is ignored. Using your own business ID (an order ID, a message ID) therefore makes retries safe.

In a cluster, webhooks and the start API work on every master (or through the load balancer). RabbitMQ, MQTT and schedule triggers run on the leader only, so each event starts exactly one run.

7. Running and controlling workflows

Every run is recorded with its status, current nodes, variables, output, incoming request data, a history of every step, and logs.

Statuses

Status	Meaning
QUEUED	Accepted; waiting for a worker with free capacity and matching tags
PENDING	Process started on a worker, about to begin
RUNNING	Executing nodes
WAITING	Every active branch is waiting on a delay, timer, signal, event or sub-workflow
PAUSED	No new node starts until resumed
COMPLETED	Finished normally
FAILED	A node failed with no error path, an End node set FAILED, or its worker was lost
STOPPED	Stopped gracefully by a user
TERMINATED	Killed by a user, or cancelled with its parent

Each step (an "activity") has its own status: RUNNING, RETRYING, COMPLETED, FAILED or CANCELLED, plus the output it took, attempt number, input, output, error and timing.

Controls

Action	Call	Effect
Stop	`POST /api/instances/<id>/stop`	Cancels waits and ends the run as STOPPED
Terminate	`POST /api/instances/<id>/terminate`	Ends it now; the process is killed after `terminateGraceMs` if needed
Pause / Resume	`POST /api/instances/<id>/pause`, `/resume`	Holds and releases new node starts
Signal	`POST /api/instances/<id>/signal` with `{"name":"approve","payload":{...}}`	Wakes a Signal node in wait mode, or interrupts through a listen-mode Signal
Retry	`POST /api/instances/<id>/retry`	Starts a new run with the same input, linked by `retryOf`

Stopping or failing a run also terminates its running sub-workflows. A signal sent before anyone waits for it is kept and delivered later.

Watching and debugging

```
curl $FF/api/instances/ORD-1001/status       # status, currentNodes, step
counts, process id
curl $FF/api/instances/ORD-1001/activities   # every step: output taken,
input, output, error, attempts
curl $FF/api/instances/ORD-1001/logs
curl $FF/api/instances/ORD-1001/request      # the stored incoming request
(auth headers hidden)
```

To find a problem: check `/status` first. QUEUED that never moves means no worker is connected or none has the workflow's tags (see `/api/cluster`). WAITING shows in `currentNodes` what it waits for. Then find the first FAILED activity and read its `error` and `input`. Function `log()` output is in `/logs`.

Logs and retention

Logs are stored per workflow and per run, with the node and master that wrote them. They are deleted after `logs.retentionHours` (default 168). Change it in the shared config, in **Settings** in the designer, or with `PUT /api/settings/log-retention`

`{"retentionHours": 72}`; in a cluster that updates every master. MongoDB enforces retention with an expiry index, etcd with a lease, and the file store with a periodic purge. `runtime.instanceRetentionHours` can also remove old finished runs.

8. REST API reference

Every master serves the full API; in a cluster, non-leader masters forward runtime calls to the leader. With `security.api.enabled`, send `Authorization: Basic ...` (or `curl -u user:pass`); `/api/health` and, unless `protectHooks` is set, `/hooks/*` stay open.

Workflows

Method and path	Purpose
`GET /api/workflows`	List workflows (`?full=1` for complete definitions)
`GET /api/workflows/:id`	One definition
`POST /api/workflows`	Create (400 with `errors` if invalid, 409 if the ID exists)
`PUT /api/workflows/:id`	Create or update; `version` goes up and triggers are re-armed on every master
`DELETE /api/workflows/:id`	Delete; running instances finish with their saved copy
`POST /api/workflows/validate`	Check without saving: `{ valid, errors }`
`POST /api/workflows/:id/start`	Body `{ instanceId?, input?, wait?: true or "finish", timeoutMs? }`
`ALL /hooks/<path>`	HTTP triggers; reply header `x-workflow-instance-id`

Instances

Method and path	Purpose
`GET /api/instances?workflowId=&status=A,B&parentInstanceId=&limit=&offset=`	List: `{ total, items }`
`GET /api/instances/:id`	Full record (`?include=all` adds activities, request and definition)

Method and path	Purpose
`GET /api/instances/:id/status`	Status, current nodes, step counts, duration, process ID
`GET /api/instances/:id/activities`	Step history (`/activities/:activityId` for one step)
`GET /api/instances/:id/request`	Stored incoming request or event
`GET /api/instances/:id/logs`	Logs for the run (`?level=warn,error&limit=`)
`GET /api/instances/:id/children`	Sub-workflow runs
`GET /api/instances/:id/wait?timeoutMs=`	Wait until the run finishes
`POST /api/instances/:id/stop` , `/terminate` , `/pause` , `/resume`	Control, body `{ reason? }`
`POST /api/instances/:id/signal`	Body `{ name, payload }`
`POST /api/instances/:id/retry`	Body `{ instanceId? }`
`DELETE /api/instances/:id`	Remove a finished run

Logs, data and events

Method and path	Purpose
`GET /api/logs?instanceId=&workflowId=&level=&since=&until=&limit=`	Search logs
`GET` / `PUT /api/settings/log-retention`	Read or set `retentionHours`
`POST /api/logs/purge`	Delete logs older than `before` (epoch ms)
`GET /api/data` , `GET /api/data/:collection?limit=`	Rows written by `db.write`
`POST /api/events/publish`	Publish a test message: `{ source: "mqtt", topic, message }` or `{ source: "rabbitmq", exchange, routingKey, message }`

Method and path	Purpose
`GET /api/stream`	Live updates as Server-Sent Events (`?instanceId=` to filter)

Cluster

Method and path	Purpose
`GET /api/health`	Node ID, whether it is leader, leader ID, term (for load balancer checks)
`GET /api/cluster`	Masters, workers (connected, capacity, load, tags, draining), running and queued counts
`GET` / `PUT /api/cluster/config`	Shared config document; `PUT` body `{ doc, version }`, needs API login
`GET /api/node-types`	Node and trigger definitions used by the designer

Each worker also answers `GET http://<worker>:<http.port>/health`: 200 when connected to a leader, 503 otherwise, with its running workflows in the body.

9. Worked examples

All four examples are in the `examples/` folder and are loaded automatically into an empty store; on an existing install add one with `curl -X POST $FF/api/workflows -H 'content-type: application/json' -d @examples/<file>.json`. Below, `$FF` is any master, for example `http://localhost:3000`.

Example 1: An external app creates an order (start API, webhook or MQTT)

Your app can start a workflow you already built without any code change in FlowForge. The order JSON becomes `input`, so nodes read `{{ input.orderId }}`, `{{ input.items }}` and so on. Use the order ID as the instance ID so a retried call cannot create a duplicate run.

Start API (works for any saved workflow):

```
curl -X POST $FF/api/workflows/order-workflow/start \
  -H 'content-type: application/json' \
  -d '{"instanceId":"ORD-1001","input":{"orderId":"ORD-1001","customer":"C-77","items":[{"sku":"A1","qty":2,"price":250}]}}'
```

From a Node.js app:

```
async function onOrderCreated(order) {
  const res = await fetch('http://flowforge:3000/api/workflows/order-
workflow/start', {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify({ instanceId: order.orderId, input: order }),
  });
  if (res.status === 409) return;              // already sent: safe to
ignore
  if (!res.ok) throw new Error(await res.text());
  return res.json();                           // { instanceId, status:
"QUEUED", ... }
}
```

Webhook: add an HTTP trigger to the workflow (`POST` , path `/orders` , `instanceIdFrom: body:orderId`), then post the plain order:

```
curl -X POST $FF/hooks/orders -H 'content-type: application/json' \
  -d '{"orderId":"ORD-1001","customer":"C-77","items":
[{"sku":"A1","qty":2,"price":250}]}'
```

MQTT: add an MQTT trigger (topic `orders/created` , `instanceIdFrom: body:orderId`), then publish to the same broker the masters use:

```
mosquitto_pub -h <broker> -t orders/created -q 1 \
  -m '{"orderId":"ORD-1001","customer":"C-77","items":
[{"sku":"A1","qty":2,"price":250}]}'
```

MQTT has no reply, so let the workflow report back with an MQTT publish node (for example to `orders/ORD-1001/status`), an HTTP activity that calls your app, or poll `GET /api/instances/ORD-1001/status` .

Example 2: Order approval with a human signal (`order-approval.json`)

A webhook receives an order; a Function node totals it; a Decision sends orders over 1000 for approval. Those reply 202 immediately through a Response node and wait for an `approve` signal (24 h timeout ends the run as FAILED). Approved and small orders are saved with `db.write` , announced on RabbitMQ (`orders` / `order.approved` , 2 retries), and answered with 200.

```
# small order: approved at once
curl -X POST $FF/hooks/orders -H 'content-type: application/json' \
  -d '{"orderId":"A-1","items":[{"price":10,"qty":2}]}'
# → {"orderId":"A-1","status":"APPROVED","total":20}

# big order: waits for approval
curl -X POST $FF/hooks/orders -H 'content-type: application/json' \
  -d '{"orderId":"A-2","items":[{"price":5000}]}'
# → 202 {"orderId":"A-2","status":"PENDING_APPROVAL","instanceId":"A-2"}

curl -X POST $FF/api/instances/A-2/signal -H 'content-type: application/json'
\
  -d '{"name":"approve","payload":{"by":"alice"}}'
curl $FF/api/instances/A-2     # status COMPLETED, output.approvedBy =
"alice"
```

Example 3: Device telemetry over HTTP or MQTT (`device-telemetry.json`)

One workflow with two triggers: `POST /hooks/devices/:deviceId/telemetry` and MQTT topic `devices/+/telemetry`. It reads the device ID from the URL or the topic (`{{ input.deviceId || trigger.topic.split('/')[1] }}`), raises an MQTT alarm above 75 degrees, saves the reading and acknowledges on `devices/<id>/ack`.

```
curl -X POST $FF/hooks/devices/pump-7/telemetry -H 'content-type:
application/json' \
  -d '{"messageId":"msg-1001","temperature":82.5}'
# → {"instanceId":"msg-1001","deviceId":"pump-7","status":"ALARM"}

MQTT_URL=mqtt://localhost:1883 node examples/clients/trigger-mqtt.js pump-9
91
# → devices/pump-9/telemetry { messageId: 'msg-…', temperature: 91 }
# ← devices/pump-9/ack { messageId: 'msg-…', status: 'ALARM' }
```

`examples/clients/trigger-http.sh` and `examples/clients/trigger-mqtt.js` are ready-to-run clients.

Example 4: Sensor alert with Switch and Delay (`sensor-alert.json`)

An MQTT reading on `sensors/+/reading` is classified as critical, warning or normal. A Switch publishes an alarm for critical readings, waits 2 seconds for warnings (debounce), and stores every reading.

```
mosquitto_pub -h <broker> -t sensors/s9/reading -m '{"value":99}'
curl "$FF/api/instances?workflowId=sensor-alert"   # output.level =
"critical"
```

Example 5: Payment saga with a sub-workflow and an interrupt (`payment-saga.json`)

A RabbitMQ message on `checkout` / `checkout.started` starts the saga, using `sagaId` as its ID. It runs `order-approval` as a sub-workflow in its own process and waits for its result, then waits up to 30 s for a `payment.confirmed` event whose `orderId` matches. A listen-mode `cancel` signal can interrupt it at any point.

```
curl -X POST $FF/api/events/publish -H 'content-type: application/json' \
  -d
'{"source":"rabbitmq","exchange":"checkout","routingKey":"checkout.started","
message":{"sagaId":"S-1","orderId":"O-77","items":[{"price":20}]}}'
curl -X POST $FF/api/events/publish -H 'content-type: application/json' \
  -d
'{"source":"rabbitmq","exchange":"payments","routingKey":"payment.confirmed",
"message":{"orderId":"O-77","amount":20}}'
curl $FF/api/instances/S-1            # COMPLETED, output.order and
output.payment

# or cancel a running saga
curl -X POST $FF/api/instances/S-2/signal -H 'content-type: application/json'
\
  -d '{"name":"cancel","payload":{"reason":"customer left"}}'
```

10. High availability and operations

Losing the leader master does not stop running workflows; losing a worker ends the workflows on it after a grace period.

What happens when something fails

Situation	What FlowForge does	What you do
Leader master crashes	Another master is elected within about one `leaseTtlMs`. Workers reconnect to it, hand over their running workflows, and replay buffered messages; event subscriptions are restored. In testing a new leader took over in about 4 s and the waiting workflow kept its process.	Restart the dead master later; it rejoins as a follower
A follower master crashes	Nothing to recover	Restart it; the load balancer routes around it meanwhile
Worker crashes	After `workerLostGraceMs` its workflows become FAILED, or are re-run from the start with `onWorkerLost: restart`	Retry failed runs with `POST /api/instances/<id>/retry` if needed
Short network drop to a worker	The worker reconnects within the grace period; nothing is lost	Nothing
Deploying a worker	Send SIGTERM: it stops taking work and waits up to `drainTimeoutMs` for running workflows	Start the new version after it exits
etcd or MongoDB unreachable	The leader steps down once its lease cannot be renewed; nothing is scheduled until it is back	Restore the coordinator; the cluster heals by itself

While no leader is reachable, a worker keeps up to `bufferLimit` messages (default 50,000). Requests that were in flight to brokers or the database when the leader died are returned as errors to the node, so its retry setting decides whether they are sent again.

Security checklist

- Set `FLOWFORGE_GRPC_PASSWORD` on every node and reference it as `${FLOWFORGE_GRPC_PASSWORD}` in the shared config; never leave the default `change-me`.

- Turn on `grpc.tls` on masters and `masters.tls.caFile` on workers across any network you don't fully trust: Basic auth sends the password with every call.
- Rotate gRPC credentials without downtime: add the new pair to `security.grpc.additionalUsers` and push, switch `username` / `password` and push, then remove the old pair.
- Turn on `security.api` for the REST API and designer, and put the masters behind a TLS load balancer (`deploy/nginx.conf`).
- Restrict access to etcd and MongoDB: they hold workflows, request data and the shared config.
- Function nodes run in Node's built-in `vm` sandbox, which is not a security boundary: only trusted people should write workflows.

Sizing

One workflow process uses about 40 to 70 MB of memory. Set each worker's `runtime.maxConcurrent` to roughly (RAM minus 1 GB) divided by 80 MB, and add workers to grow. Use MongoDB for the store and `db.write` data at high volume; keep etcd for coordination, the shared config and small data sets.

Troubleshooting

Symptom	Likely cause	Fix
Runs stay QUEUED	No worker connected, all full, or no worker has the workflow's tags	Check `GET /api/cluster`; add a worker or capacity, or fix `settings.workerTags`
`UNAUTHENTICATED` in logs	gRPC credentials differ between nodes	Align `security.grpc` and each node's `FLOWFORGE_GRPC_PASSWORD`
503 "no leader elected yet"	Coordinator unreachable or an election in progress	Check etcd or MongoDB; retry shortly
MQTT or RabbitMQ messages start nothing	Masters not connected to that broker, or the trigger is disabled	Set `mqtt.url` / `rabbitmq.url` in the shared config and push
409 when starting	That instance ID already exists	Expected for duplicates; use a new ID or retry the old run
Shared config push refused (409)	Someone changed it after you pulled	Pull, re-apply your change, push again

Symptom	Likely cause	Fix
Node waits at startup for shared config	Nothing pushed yet	Run `config push`; nodes wait up to `shared.waitMs` (30 s)

Known limits

- A workflow's state lives in its own process. It survives a leader failover but not the loss of its worker; resuming from a checkpoint is not built.
- During a network split, two masters can both act as leader for up to one lease period.
- Keeping `db.write` data in etcd suits small tables; use MongoDB for real volumes.
- Tested in the build environment: single node, separate master and worker, the etcd cluster (failover, worker loss, drain, shared config changes) and gRPC over TLS. The MongoDB store, coordinator and shared config, and real RabbitMQ, are implemented but were not tested there; MQTT was tested against a real broker.